CHAPTER

5

Delay Calculation

This chapter provides an overview of the delay calculation of cell-based designs for the pre-layout and post-layout timing verification. The previous chapters have focused on the modeling of the interconnect and the cell library. The cell and interconnect modeling techniques are utilized to obtain timing of the design.

5.1 Overview

5.1.1 Delay Calculation Basics

A typical design comprises of various combinational and sequential cells. We use an example logic fragment shown in Figure 5-1 to describe the concept of delay calculation.

J. Bhasker and R. Chadha, *Static Timing Analysis for Nanometer Designs: A Practical Approach*,
DOI: 10.1007/978-0-387-93820-2_5, © Springer Science + Business Media, LLC 2009

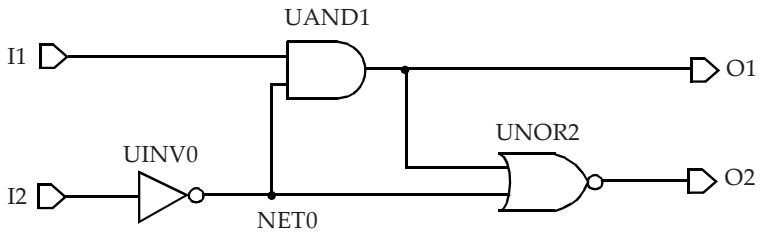


Figure 5-1 *Schematic of an example logic block.*

The library description of each cell specifies the pin capacitance values for each of the input pins[1]. Thus, every net in the design has a capacitive load which is the sum of the pin capacitance loads of every fanout of the net plus any contribution from the interconnect. For the purposes of simplicity, the contributions from the interconnect are not considered in this section - those are described in the later sections. Without considering the interconnect parasitics, the internal net *NET0* in Figure 5-1 has a net capacitance which is comprised of the input pin capacitances from the *UAND1* and *UNOR2* cells. The output *O1* has the pin capacitance of the *UNOR2* cell plus any capacitive loading for the output of the logic block. Inputs *I1* and *I2* have pin capacitances corresponding to the *UAND1* and *UINV0* cells. With such an abstraction, the logic design in Figure 5-1 can be described by an equivalent representation shown in Figure 5-2.

As described in Chapter 3, the cell library contains NLDM timing models for various timing arcs. The non-linear models are represented as two-dimensional tables in terms of input transition time and output capacitance. The output transition time of a logic cell is also described as a two-dimensional table in terms of input transition and total output capacitance at the net. Thus, if the input transition time (or slew) is specified at the inputs of the logic block, the output transition time and delays through the timing arcs of the *UINV0* cell and *UAND1* cell (for the input *I1*) can be obtained from the library cell descriptions. Extending the same approach through the fanout cells, the transition times and delays through the other timing

1. The standard cell libraries normally do not specify pin capacitances for cell outputs.

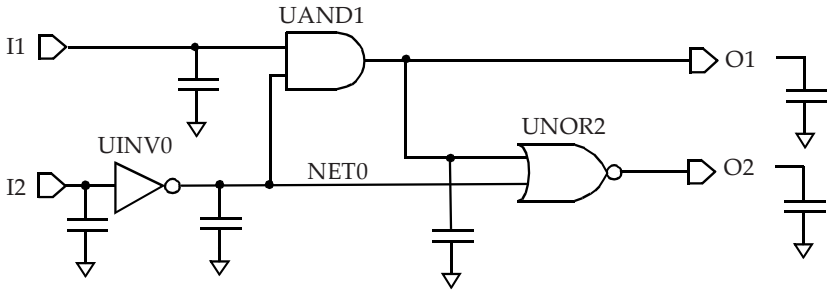


Figure 5-2 *Logic block representation depicting capacitances.*

arc of the *UAND1* cell (from *NET0* to *O1*) and through the *UNOR2* cell can be obtained. For a multi-input cell (such as *UAND1*), different input pins can provide different values of output transition times. The choice of transition time used for the fanout net depends upon the slew merge option and is described in Section 5.4. Using the approach described above, the delays through any logic cell can be obtained based upon the transition time at the input pins and the capacitance present at the output pins.

5.1.2 Delay Calculation with Interconnect

Pre-layout Timing

As described in Chapter 4, the interconnect parasitics are estimated using wireload models during the pre-layout timing verification. In many cases, the resistance contribution in the wireload models is set to 0. In such scenarios, the wireload contribution is purely capacitive and the delay calculation methodology described in the previous section is applicable to obtain the delays for all the timing arcs in the design.

In cases where the wireload models include the effect of the resistance of the interconnect, the NLDM models are used with the total net capacitance for the delay through the cell. Since the interconnect is resistive, there is additional delay from the output of the driving cell to the input pin of the

fanout cell. The interconnect delay calculation process is described in Section 5.3.

Post-layout Timing

The parasitics of the metal traces map into an RC network between driver and destination cells. Using the example of Figure 5-1, the interconnect resistance of the nets is illustrated in Figure 5-3. An internal net such as *NET0* in Figure 5-1 maps into multiple subnodes as shown in Figure 5-3. Thus, the output load of the inverter cell *UINV0* is comprised of an RC structure. Since the NLDM tables are in terms of input transition and output capacitance, the resistive load at the output pin implies that the NLDM tables are not directly applicable. Using the NLDM with resistive interconnect is described in the next section.

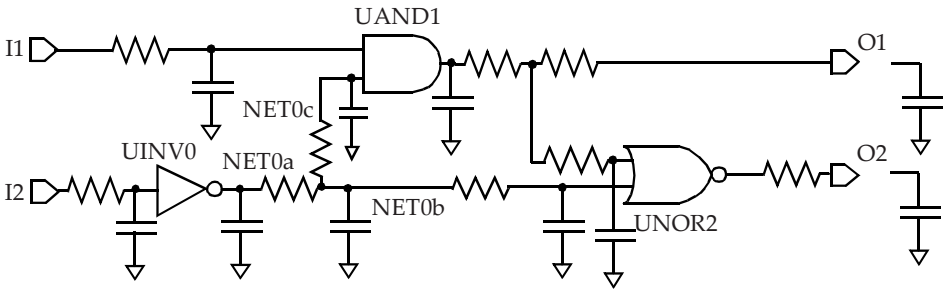


Figure 5-3 *Logic block representation depicting resistive nets.*

5.2 Cell Delay using Effective Capacitance

As described above, the NLDM models are not directly usable when the load at the output of the cell includes the interconnect resistance. Instead, an "effective" capacitance approach is employed to handle the effect of resistance.

The effective capacitance approach attempts to find a single capacitance that can be utilized as the equivalent load so that the original design as well the design with equivalent capacitance load behave similarly in terms of timing at the output of the cell. This equivalent single capacitance is termed as **effective capacitance**.

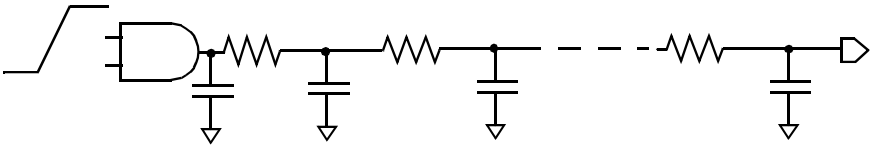
Figure 5-4(a) shows a cell with an RC interconnect at its fanout. The RC interconnect is represented by an equivalent RC PI-network shown in Figure 5-4(b). The concept of effective capacitance is to obtain an equivalent output capacitance *Ceff* (shown in Figure 5-4(c)) which has the same delay through the cell as the original design with the RC load. In general, the cell output waveform with the RC load is very different from the waveform with a single capacitive load.

Figure 5-5 shows representative waveforms at the output of the cell with total capacitance, effective capacitance and the waveform with the actual RC interconnect. The effective capacitance *Ceff* is selected so that the delay (as measured at the midpoint of the transition) at the output of the cell in Figure 5-4(c) is the same as the delay in Figure 5-4(a). This is illustrated in Figure 5-5.

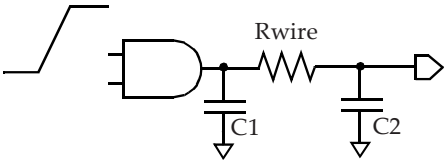
In relation to the PI-equivalent representation, the effective capacitance can be expressed as:

```
Ceff = C1 + k * C2, 0 <= k <= 1
```

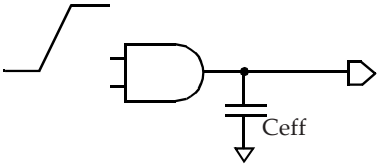
where *C1* is the near-end capacitance and *C2* is the far-end capacitance as shown in Figure 5-4(b). The value of *k* lies between 0 and 1. In the scenario where the interconnect resistance is negligible, the effective capacitance is nearly equal to the total capacitance. This is directly explainable by setting *R* to 0 in Figure 5-4(b). Similarly, if the interconnect resistance is relatively large, the effective capacitance is almost equal to the near-end capacitance *C1* (Figure 5-4(b)). This can be explained by increasing *R* to the limiting case where *R* becomes infinite (essentially an open circuit).



(a) RC interconnect.



(b) PI model.



(c) Effective capacitance.

Figure 5-4 *RC interconnect with effective capacitance.*

The effective capacitance is a function of:

i. the driving cell, and

ii. the characteristics of the load or specifically the input impedance of the load as seen from the driving cell.

For a given interconnect, a cell with a weak output drive will see a larger effective capacitance than a cell with a strong drive. Thus, the effective capacitance will be between the minimum value of *C1* (for high interconnect resistance or strong driving cell) and the maximum value which is the

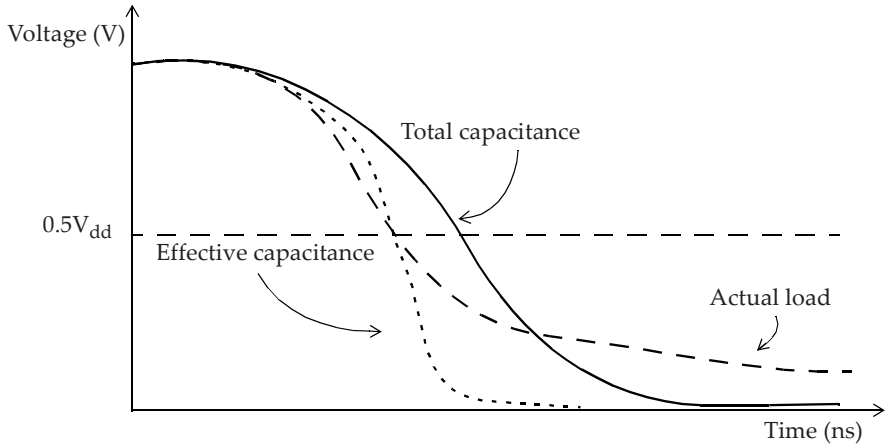


Figure 5-5 *Waveform at the output of the cell with various loads.*

same as the total capacitance when the interconnect resistance is negligible or the driving cell is weak. Note that the destination pin transitions later than the output of the driving cell. The phenomenon of near-end capacitance charging faster than the far-end capacitance is also referred to as the *resistive shielding effect* of the interconnect, since only a portion of the far-end capacitance is seen by the driving cell.

Unlike the computation of the delay by direct lookup of the NLDM models in the library, the delay calculation tools obtain the effective capacitance by an iterative procedure. In terms of the algorithm, the first step is to obtain the driving point impedance seen by the cell output for the actual RC load. The driving point impedance for the actual RC load is calculated using any of the methods such as second order AWE or Arnoldi algorithms[1]. The next step in computing effective capacitance is equating the charge transferred until the midpoint of the transition in the two scenarios. The charge transferred at the cell output when using the actual RC load (based upon driving point impedance) is matched with the charge transfer when using

1. See [ARN51] in Bibliography.

the effective capacitance as the load. Note that the charge transfer is matched only until the midpoint of the transition. The procedure starts with an estimate of the effective capacitance and then iteratively updates the estimate. In most practical scenarios, the effective capacitance value converges within a small number of iterations.

The effective capacitance approximation is thus a good model for computing delay through the cell. However, the output slew obtained using effective capacitance does not correspond to the actual waveform at the cell output. The waveform at the cell output, especially for the trailing half of the waveform, is not represented by the effective capacitance approximation. Note that in a typical scenario, the waveform of interest is not at the cell output but at the destination points of the interconnect which are the input pins of the fanout cells.

There are various approaches to compute the delay and the waveform at the destination points of the interconnect. In many implementations, the effective capacitance procedure also computes an equivalent Thevenin voltage source for the driving cell. The Thevenin source comprises of a ramp source with a series resistance *Rd* as shown in Figure 5-6. The series resistance *Rd* corresponds to the pull-down (or pull-up) resistance of the output stage of the cell.

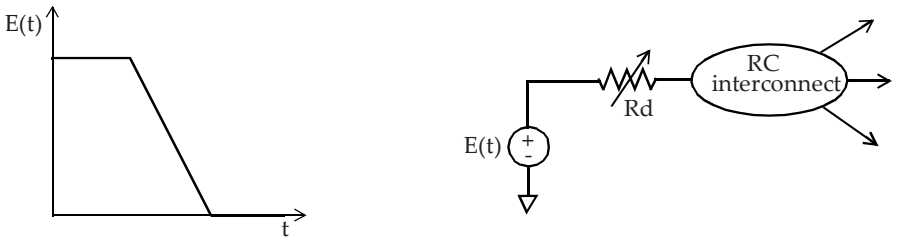


Figure 5-6 *Thevenin source model for driving cell.*

This section described the computation of the delay through the driving cell with an RC interconnect using the effective capacitance. The effective capacitance computation also provides the equivalent Thevenin source

model which is then used to obtain the timing through the RC interconnect. The process of obtaining timing through the interconnect is described next.

5.3 Interconnect Delay

As described in Chapter 4, the interconnect parasitics of a net are normally represented by an RC circuit. The RC interconnect can be pre-layout or post-layout. While the post-layout parasitic interconnect can include coupling to neighboring nets, the basic delay calculation treats all capacitances (including coupling capacitances) as capacitances to ground. An example of parasitics of a net along with its driving cell and a fanout cell is shown in Figure 5-7.

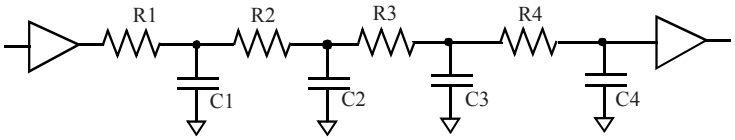


Figure 5-7 *Parasitics of a net.*

Using the effective capacitance approach, the delay through the driving cell and through the interconnect are obtained separately. The effective capacitance approach provides the delay through the driving cell as well as the equivalent Thevenin source at the output of the cell. The delay through the interconnect is then computed separately using the Thevenin source. The interconnect portion has one input and as many outputs as the destination pins. Using the equivalent Thevenin voltage source at the input of the interconnect, the delays to each of the destination pins are computed. This is illustrated in Figure 5-6.

For pre-layout analysis, the RC interconnect structure is determined by the tree type, which in turn determines the net delay. The three types of interconnect tree representations are described in detail in Section 4.2. The selected tree type is normally defined by the library. In general, the worst-

case slow library would select the worst-case tree type since that tree type provides the largest interconnect delay. Similarly, the best-case tree structure, which does not include any resistance from the source pin to the destination pins, is normally selected for the best-case fast corner. The interconnect delay for the best-case tree type is thus equal to zero. The interconnect delay for the typical-case tree and worst-case tree are handled just like for the post-layout RC interconnect.

Elmore Delay

Elmore delays are applicable for RC trees. What is an RC tree? An **RC tree** meets the following three conditions:

- Has a single input (source) node.
- Does not have any resistive loops.
- All capacitances are between a node and ground.

Elmore delay can be considered as finding the delay through each segment, as the *R* times the downstream capacitance, and then taking the sum of the delays from the root to the sink.

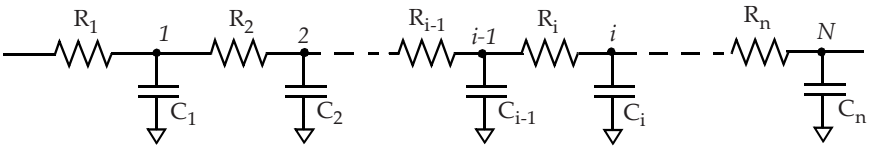


Figure 5-8 *Elmore delay model.*

The delays to various intermediate nodes are represented as:

```
Td1 = C1 * R1;
Td2 = C1 * R1 + C2 * (R1 + R2);
. . .
Tdn = Σ(i=1,N) Ci (Σ(j=1,i) Rj); # Elmore delay equation
```

Elmore delay is mathematically identical to considering the first moment of the impulse response[1]. We now apply the Elmore delay model to a simplified representation of a wire with R_{wire} and C_{wire} as the parasitics, plus a load capacitance C_{load} to model the pin capacitance at the far-end of the wire. The equivalent RC network can be simplified as a PI network model or a T-representation as shown in Figure 4-4 and Figure 4-3 respectively. Either representation provides the net delay (based upon Elmore delay equation) as:

$$R_{wire} * (C_{wire} / 2 + C_{load})$$

This is because the C_{load} sees the entire wire resistance in its charging path, whereas the C_{wire} capacitance sees $R_{wire} / 2$ for the T-representation, and $C_{wire} / 2$ sees R_{wire} in its charging path for the PI-representation. The above approach can be extended to a more complex interconnect structure also.

An example of Elmore delay calculation for a net using wireload model with balanced RC tree (and also worst-case RC tree) is given below.

Using a balanced tree model, the resistance and capacitance of a net is divided equally among the branches of the net (assuming a fanout of *N*). For a branch with pin load *Cpin*, the delay using balanced tree is:

$$\text{net delay} = (R_{wire} / N) * (C_{wire} / (2 * N) + C_{pin})$$

In the worst-case tree model, the resistance and entire capacitance of the net is considered for each endpoint of the net. Here *Cpins* is the total pin load from all fanouts:

$$\text{Net delay} = R_{wire} * (C_{wire} / 2 + C_{pins})$$

Figure 5-9 shows an example design.

1. See [RUB83] in Bibliography.

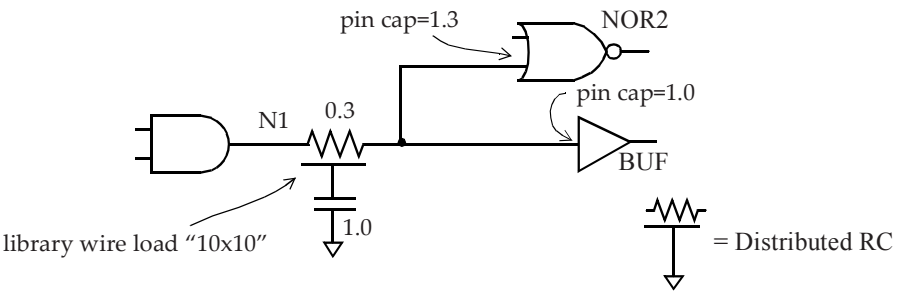


Figure 5-9 *Net delay for worst-case tree using Elmore model.*

If we use the worst-case tree model to calculate the delay for net *N1*, we get:

$$\text{Net delay} = R_{wire} * (C_{wire}/2 + C_{pins})$$
$$= 0.3 * (0.5 + 2.3) = 0.84$$

If we use the balanced tree model, we get the following delays for the two branches of the net *N1*:

$$\text{Net delay to NOR2 input pin} = (0.3/2) * (0.5/2 + 1.3)$$
$$= 0.2325$$

$$\text{Net delay to BUF input pin} = (0.3/2) * (0.5/2 + 1.0)$$
$$= 0.1875$$

Higher Order Interconnect Delay Estimation

As described above, the Elmore delay is the first moment of the impulse response. The AWE (Asymptotic Waveform Evaluation), Arnoldi or other methods match higher order moments of response. By considering the higher order estimates, greater accuracy for computing the interconnect delays is obtained.

Full Chip Delay Calculation

So far, this chapter has described the computation of delays for a cell and the interconnect at the output of a cell. Thus, given the transition at the inputs of the cell, the delays through the cell and the interconnect at the output of the cell can be computed. The transition time at the far-end of the interconnect (destination or sink point) is the input to the next stage and this process is repeated throughout the entire design. The delays for each timing arc in the design are thus calculated.

5.4 Slew Merging

What happens when multiple slews arrive at a common point, such as in the case of a multi-input cell or a multi-driven net? Such a common point is referred to as a *slew merge point*. Which slew is chosen to propagate forward at the slew merge point? Consider the 2-input cell shown in Figure 5-10.

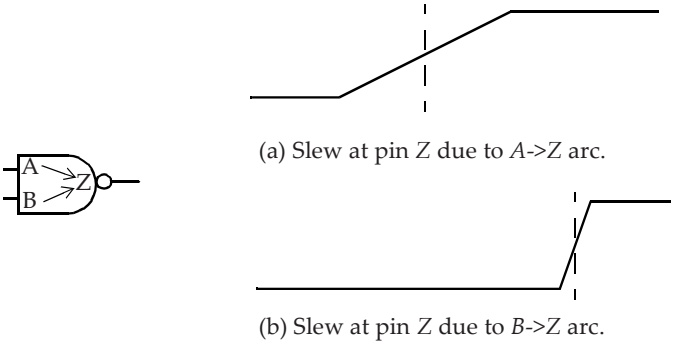


Figure 5-10 *Slews at merge point.*

The slew at pin *Z* due to signal changing on pin *A* arrives early but is slow to rise (slow slew). The slew at pin *Z* due to signal changing on pin *B* arrives late but is fast to rise (fast slew). At the slew merge point, such as pin

Z, which slew should be selected for further propagation? Either of these slew values may be correct depending upon the type of timing analysis (max or min) being performed as described below.

There are two possibilities when doing max path analysis:

i. *Worst slew propagation*: This mode selects the worst slew at the merge point to propagate. This would be the slew in Figure 5-10(a). For a timing path that goes through pins *A*->*Z*, this selection is exact, but is pessimistic for any timing path that goes through pins *B*->*Z*.

ii. *Worst arrival propagation*: This mode selects the worst arrival time at the merge point to propagate. This corresponds to the slew in Figure 5-10(b). The slew chosen in this case is exact for a timing path that goes through pins *B*->*Z* but is optimistic for a timing path that goes through pins *A*->*Z*.

Similarly, there are two possibilities when doing min path analysis:

i. *Best slew propagation*: This mode selects the best slew at the merge point to propagate. This would be the slew in Figure 5-10(b). For a timing path that goes through pins *B*->*Z*, this selection is exact, but the selection is smaller for any timing path that goes through pins *A*->*Z*. For the paths going through *A*->*Z*, the path delays are smaller than the actual values and is thus pessimistic for min path analysis.

ii. *Best arrival propagation*: This mode selects the best arrival time at the merge point to propagate. This corresponds to the slew in Figure 5-10(a). The slew chosen in this case is exact for a timing path that goes through pins *A*->*Z* but the selection is larger than the actual values for a timing path that goes through pins *B*->*Z*. For the paths going through *B*->*Z*, the path delays are larger than the actual values and is thus optimistic for min path analysis.

A designer may perform delay calculation outside of the static timing analysis environment for generating an SDF. In such cases, the delay calculation tools normally use the worst slew propagation. The resulting SDF is adequate for max path analysis but may be optimistic for min path analysis.

Most static timing analysis tools use worst and best slew propagation as their default as it bounds the analysis by being conservative. However, it is possible to use the exact slew propagation when a specific path is being analyzed. The exact slew propagation may require enabling an option in the timing analysis tool. Thus, it is important to understand what slew propagation mode is being used by default in a static timing analysis tool, and understand situations when it may be overly pessimistic.

5.5 Different Slew Thresholds

In general, a library specifies the slew (transition time) threshold values used during characterization of the cells. The question is, what happens when a cell with one set of slew thresholds drives other cells with different set of slew threshold settings? Consider the case shown in Figure 5-11 where a cell characterized with 20-80 slew threshold drives two fanout cells; one with a 10-90 slew threshold and the other with a 30-70 slew threshold and a slew derate of 0.5.

The slew settings for cell *U1* are defined in the cell library as follows:

```
slew_lower_threshold_pct_rise : 20.00
slew_upper_threshold_pct_rise : 80.00

slew_derate_from_library : 1.00
input_threshold_pct_fall : 50.00
output_threshold_pct_fall : 50.00
input_threshold_pct_rise : 50.00
output_threshold_pct_rise : 50.00
```

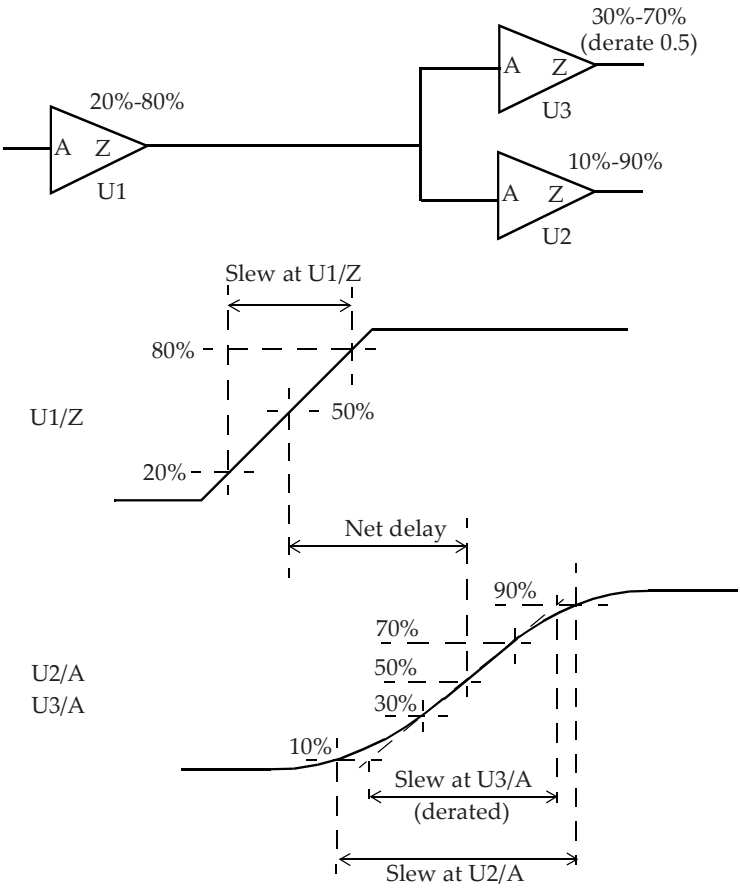


Figure 5-11 *Different slew trip points.*

```
slew_lower_threshold_pct_fall : 20.00
slew_upper_threshold_pct_fall : 80.00
```

The cell *U2* from another library can have the slew settings defined as:

```
slew_lower_threshold_pct_rise : 10.00
slew_upper_threshold_pct_rise : 90.00

slew_derate_from_library : 1.00
slew_lower_threshold_pct_fall : 10.00
slew_upper_threshold_pct_fall : 90.00
```

The cell *U3* from yet another library can have the slew settings defined as:

```
slew_lower_threshold_pct_rise : 30.00
slew_upper_threshold_pct_rise : 70.00

slew_derate_from_library : 0.5
slew_lower_threshold_pct_fall : 30.00
slew_upper_threshold_pct_fall : 70.00
```

Only the slew related settings for *U2* and *U3* are shown above; the delay related settings for input and output thresholds are 50% and not shown above. The delay calculation tools compute the transition times according to the slew thresholds of the cells connecting the net. Figure 5-11 shows how the slew at *U1/Z* corresponds to the switching waveform at this pin. The equivalent Thevenin source at *U1/Z* is utilized to obtain the switching waveforms at the inputs of the fanout cells. Based upon the waveforms at *U2/A* and *U3/A* and their slew thresholds, the delay calculation tools compute the slews at *U2/A* and at *U3/A*. Note that the slew at *U2/A* is based upon 10-90 settings whereas the slew used for *U3/A* is based upon 30-70 settings which is then derated based upon the *slew_derate* of 0.5 as specified in the library. This example illustrates how the slew at the input of the fanout cell is computed based upon the switching waveform and the slew threshold settings of the fanout cell.

During the pre-layout design phase where the interconnect resistance may not be considered, the slews at a net with different thresholds can be com-

puted in the following manner. For example, the relationship between the 10-90 slew and the 20-80 slew is:

```
slew2080 / (0.8 - 0.2) = slew1090 /(0.9 - 0.1)
```

Thus, a slew of 500ps with 10-90 measurement points corresponds to a slew of (500ps * 0.6) / 0.8 = 375ps with 20-80 measurement points. Similarly, a slew of 600ps with 20-80 measurement points corresponds to a slew of (600ps * 0.8) / 0.6 = 800ps with 10-90 measurement points.

5.6 Different Voltage Domains

A typical design may use different power supply levels for different portions of the chip. In such cases, level shifting cells are used at the interface between different power supply domains. A level shifting cell accepts input at one supply domain and provides output at the other supply domain. As an example, a standard cell input can be at 1.2V and its output can be at a reduced power supply, which may be 0.9V. Figure 5-12 shows an example.

Notice that the delay is calculated from the 50% threshold points. These points are at different voltages for different pins of the interface cells.

5.7 Path Delay Calculation

Once all the delays for each timing arc are available, the timing through the cells in the design can be represented as a timing graph. The timing through the combinational cells can be represented as timing arcs from inputs to outputs. Similarly, the interconnect is represented with corresponding arcs from the source to each destination (or sink) point represented as a separate timing arc. Once the entire design is annotated by corresponding arcs, computing the path delay involves adding up all the net and cell timing arcs along the path.

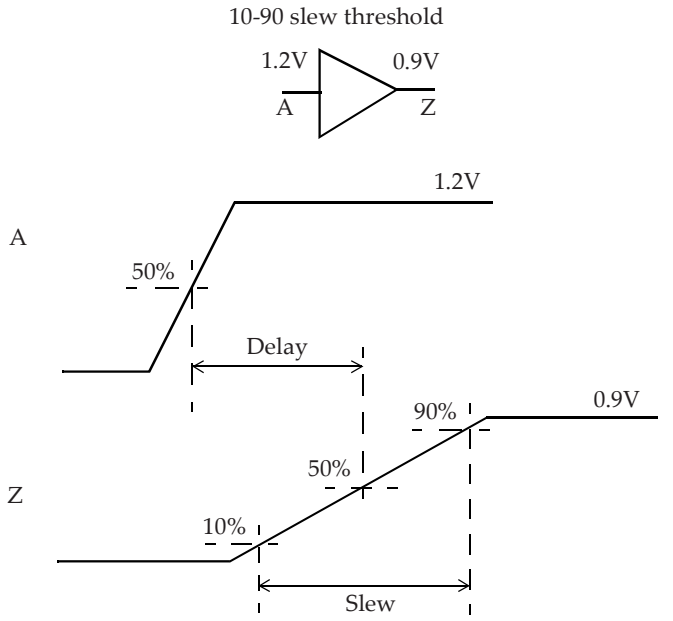


Figure 5-12 *Cell with different input and output voltages.*

5.7.1 Combinational Path Delay

Consider the three inverters in series as shown in Figure 5-13. While considering paths from net *N0* to net *N3*, we consider both rising edge and falling edge paths. Assume that there is a rising edge at net *N0*.

The transition time (or slew) at the input of the first inverter may be specified; in the absence of such a specification, a transition time of 0 (corresponding to an ideal step) is assumed. The transition time at the input *UINVa/A* is determined by using the interconnect delay model as specified in the previous section. The same delay model is also used in determining the delay, *Tn0*, for net *N0*.

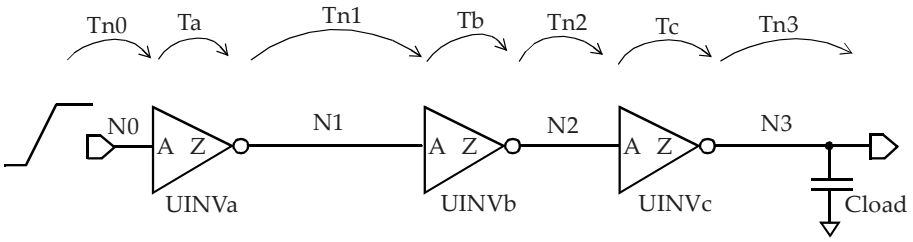


Figure 5-13 *Timing a combinational path.*

The effective capacitance at the output *UINVa/Z* is obtained based upon the RC load at the output of *UINVa*. The transition time at input *UINVa/A* and the equivalent effective load at output *UINVa/Z* is then used to obtain the cell output fall delay.

The equivalent Thevenin source model at pin *UINVa/Z* is used to determine the transition time at pin *UINVb/A* by using the interconnect model. The interconnect model is also used to determine the delay, *Tn1*, on the net *N1*.

Once the transition time at input *UINVb/A* is known, the process for calculating the delay through *UINVb* is similarly utilized. The RC interconnect at *UINVb/Z*, and the pin capacitance of pin *UINVc/A* are used to determine the effective load at *N2*. The transition time at *UINVb/A* is used to determine the rise delay through the inverter *UINVb*, and so on.

The load at the last stage is determined by any explicit load specification provided, or in the absence of which only the wire load of net *N3* is used.

The above analysis assumed a rising edge at net *N0*. Similar analysis can be carried out for the case of a falling edge on net *N0*. Thus, in this simple example, there are two timing paths with the following delays:

$$T_{fall} = Tn0_{rise} + Ta_{fall} + Tn1_{fall} + Tb_{rise} + Tn2_{rise} + Tc_{fall} + Tn3_{fall}$$

$$T_{rise} = Tn0_{fall} + Ta_{rise} + Tn1_{rise} + Tb_{fall} + Tn2_{fall} + Tc_{rise} + Tn3_{rise}$$

In general, the rise and fall delays through the interconnect can be different because of different Thevenin source models at the output of the driving cell.

5.7.2 Path to a Flip-flop

Input to Flip-flop Path

Consider the timing of the path from input *SDT* to flip-flop *UFF1* as shown in Figure 5-14.

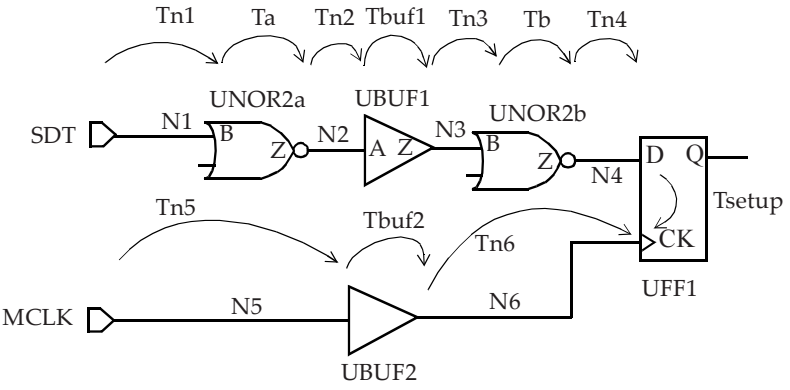


Figure 5-14 *Path to a flip-flop.*

We need to consider both rising edge and falling edge paths. For the case of a rising edge on input *SDT,* the data path delay is:

$$Tn1_{rise} + Ta_{fall} + Tn2_{fall} + Tbuf1_{fall} + Tn3_{fall} + Tb_{rise} + Tn4_{rise}$$

Similarly, for a falling edge on input *SDT*, the data path delay is:

$$Tn1_{fall} + Ta_{rise} + Tn2_{rise} + Tbuf1_{rise} + Tn3_{rise} + Tb_{fall} + Tn4_{fall}$$

The capture clock path delay for a rising edge on input *MCLK* is:

$$Tn5_{rise} + Tbuf2_{rise} + Tn6_{rise}$$

Flip-flop to Flip-flop Path

An example of a data path between two flip-flops and corresponding clock paths is shown in Figure 5-15.

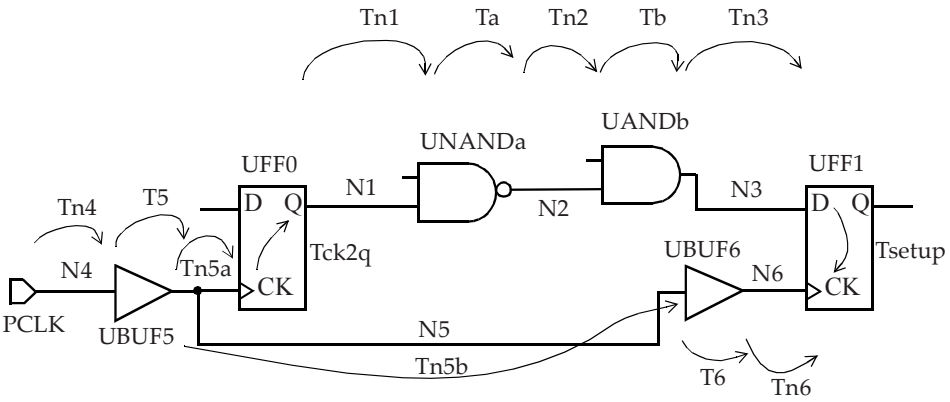


Figure 5-15 *Flip-flop to flip-flop.*

The data path delay for a rising edge on *UFF0/Q* is:

$$Tck2q_{rise} + Tn1_{rise} + Ta_{fall} + Tn2_{fall} + Tb_{fall} + Tn3_{fall}$$

The launch clock path delay for a rising edge on input *PCLK* is:

$$Tn4_{rise} + T5_{rise} + Tn5a_{rise}$$

The capture clock path delay for a rising edge on input *PCLK* is:

$$Tn4_{rise} + T5_{rise} + Tn5b_{rise} + T6_{rise} + Tn6_{rise}$$

Note that unateness of the cell needs to be considered as the edge direction may change as it goes through a cell.

5.7.3 Multiple Paths

Between any two points, there can be many paths. The longest path is the one that takes the longest time; this is also called the worst path, a late path or a max path. The shortest path is the one that takes the shortest time; this is also called the best path, an early path or a min path.

See the logic and the delays through the timing arcs in Figure 5-16. The longest path between the two flip-flops is through the cells *UBUF1*, *UNOR2*, and *UNAND3*. The shortest path between the two flip-flops is through the cell *UNAND3*.

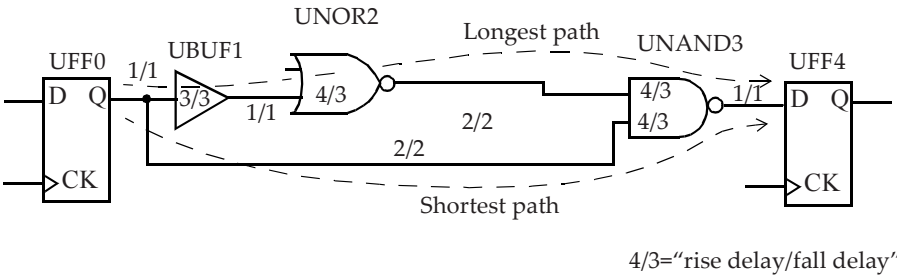


Figure 5-16 *Longest and shortest paths.*

5.8 Slack Calculation

Slack is the difference between the required time and the time that a signal arrives. In Figure 5-17, the data is required to be stable at time 7ns for the setup requirement to be met. However data becomes stable at 1ns. Thus, the slack is 6ns (= 7ns - 1ns).

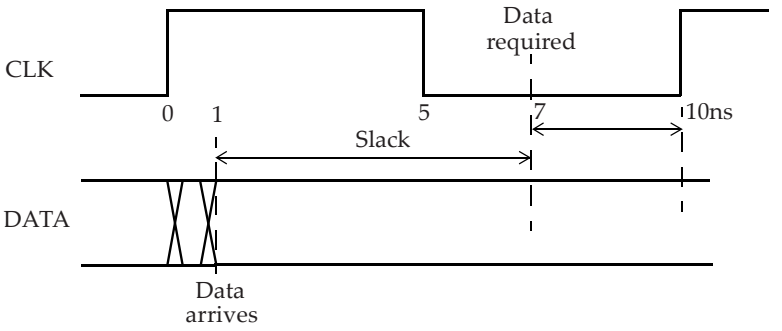


Figure 5-17 *Slack.*

Assuming that the data required time is obtained from the setup time of a capture flip-flop,

```
Slack = Required_time - Arrival_time
Required_time = Tperiod - Tsetup(capture_flip_flop)
              = 10 - 3 = 7ns
Arrival_time = 1ns
Slack = 7 - 1 = 6ns
```

Similarly if there is a skew requirement of 100ps between two signals and the measured skew is 60ps, the slack in skew is 40ps (= 100ps - 60ps).

❑